Chapter 12
Selecting Features

A map document can contain multiple maps. A map in a document is also known as a data frame. Section 1 will show you how to manipulate multiple map objects in a document in ArcMap. Section 2 will introduce how to perform definition query. A definition query is to use a query expression to select a subset of features from a feature layer to display and filter out other features that do not satisfy the query criteria. Query expression will be explained and discussed in this section. Section 3 will show you how to select features from a feature layer by using query expressions. Section 4 will introduce two ArcObjects container components that are used to hold selected features or objects: selection set and cursor. In ArcMap, a selection set holds objects selected by any means including interactive selection using the select tool and querying attributes. A selection set is an unordered pile. It does not support retrieval of individual objects from it. A cursor is similar to tube in which objects are ordered. It support retrieval of individual objects from it. This section will explain different ways to access these two container objects and manipulate objects in them. Section 5 will introduce the feature cursor component that inherits the cursor class. A feature cursor object is used to hold selected features and it supports retrieval of individual features. Once a feature is retrieved, its spatial and attribute data can be accessed. Once you are able to access the spatial and attribute data of individual features, you can master the power of GIS.

1. Maps

A map document can maintain multiple maps (data frames) each having its own spatial reference and multiple layers. In Figure 1, the map document (Flagstaff.mxd) contains three maps: Land, Transportation and, Utility, in which Land is the active map. You can manually activate any map in the document by right clicking on the map title, e.g. Transportation, in the TOC and select “Activate” from the context menu.

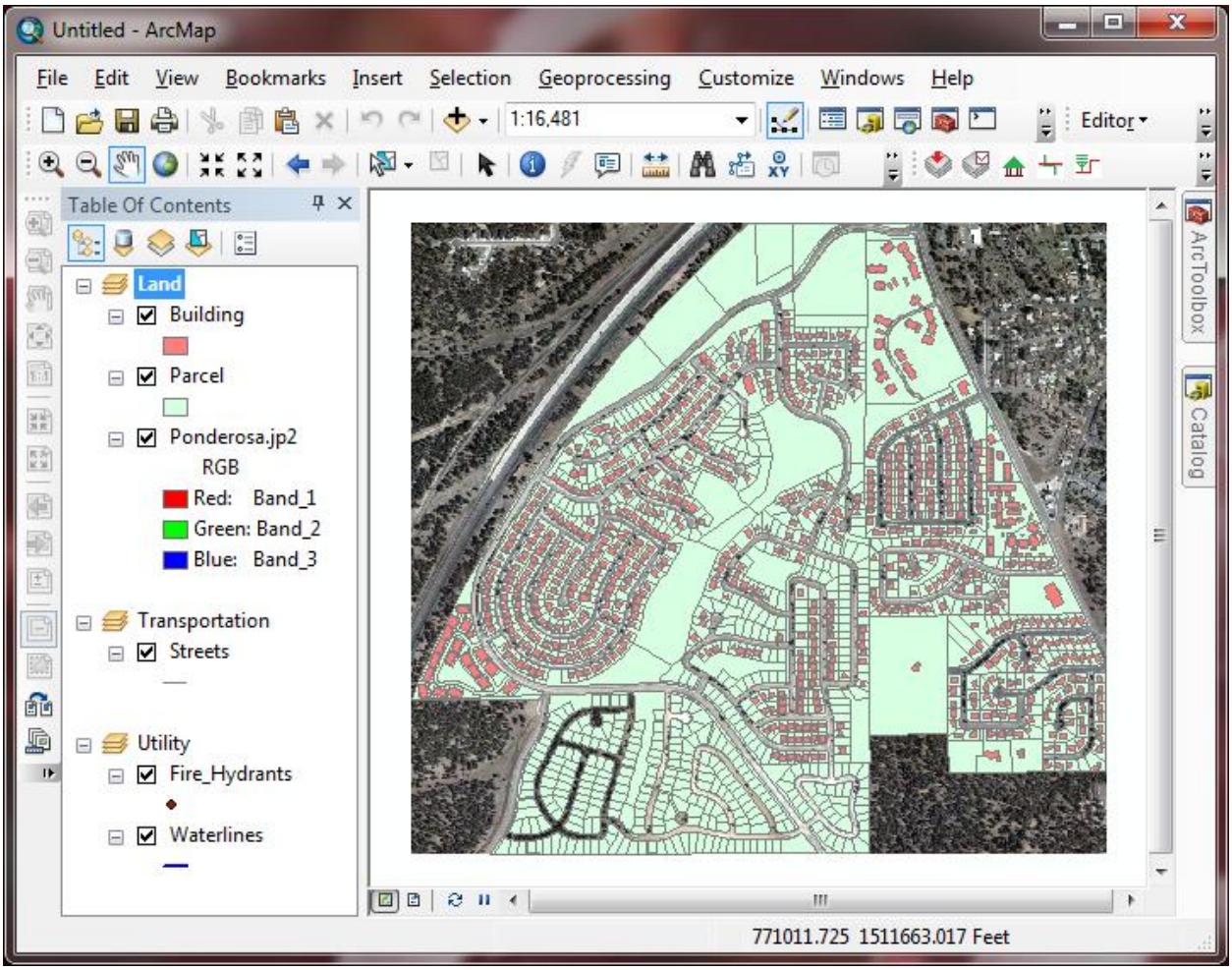


Figure 1 Multiple maps in one map document

So far, we have been using the active map (focus map) only in examples and exercises in this book. When you added a layer into the ArcMap, the layer goes to the only map with a default title “Layers”. You will learn how to create multiple maps in one document and how to access layers in different maps in this section.

You have learned how to hop from the map document (MxDocument) to the focus map with the aid of object model diagrams. Figure 2 further explains how to access any map in a map document containing multiple maps. On the IMxDocument interface of the MxDocument class, property Maps returns a maps object with IMaps interface. A maps object is a container that holds all map objects managed by the document including the focus map. You can get any map object from this container and add map objects into it.

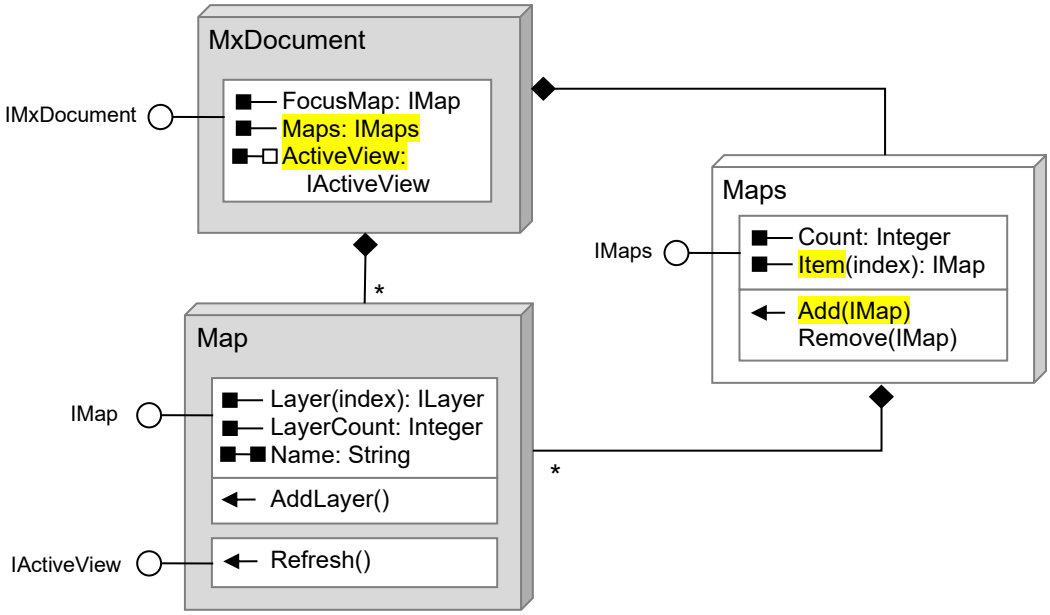


Figure 2 Relationships between MxDocument, Maps, and Map

1.1 Setting a name to the focus map

When you click the add layers button in your CityGIS toolbar, all layers are added into the only active map with a default name “Layers”. To change the default name of this map to “Land” by code, please perform the following actions:

1) Open your CityGIS add-in project if it is not loaded.
2) In the AddLayersButton.vb class module, find the OnClick event procedure
3) Below the line that gets the focus map object, i.e. `Dim map As IMap = mxDoc.FocusMap`, add a new line `map.Name = "Land"`
4) Run the program and click the add layers button again. You will see the result.

1.2 Creating new map objects in a map document

Example 1: In the CityGIS project, suppose we want to keep the satellite image, parcel, and building feature classes in the “Land” map and would like to put the water line and fire hydrants layers in another map named “Utility”, then we must instantiate a new map object from the Map co-class and add it into the maps object. Please add the following three lines of code just above the line that calls the CreateShapeLayer() function to create the waterline layer from a shapefile in the OnClick event procedure of the add layers button.

```
' add a Utility map (data frame)
map = New Map           ' creates a new map object from the Map co-class
map.Name = "Utility"    ' sets the Name property
mxDoc.Maps.Add(map)     ' adds the map into the Maps container
```

We do not need to use the Dim keyword to declare the *map* variable since it has been declared in this procedure. We can reuse the variable by assigning a new map object to it. After adding

the above new code, run the program and click the add layers button, you should find two maps, Land and Utility, appear in the map document.

We can continue to create a new map named “Transportation” and add the streets feature class (in the personal geodatabase on the CD-ROM) into it. To do so, please insert the following block of code just above the line that calls the CreateFeatureLayer() to create the streets layer in the OnClick event procedure.

```
' create a transportation map
map = New Map                ' creates a new map object from the Map co-class
map.Name = "Transportation"  ' sets the Name property
mxDoc.Maps.Add(map)          ' adds it into the Maps container
```

Now you can run the program and click the add layers button in the CityGIS toolbar. You should find three maps appear in ArcMap looking similar to Figure 1.

1.3 Accessing and activating a map

Activating a map can be carried out by assigning a map object (with IActiveView interface) to the ActiveView property on the map document (IMxDocument interface). To do so, you need to get a map object from the maps container first. The maps object has an Item property that returns a map object of a specified index. Map objects in the maps container are ordered from top to bottom. Therefore, the index of the “Land” map is 0, the “Transportation” map is 1, and the “Utility” map is 2 in the CityGIS project. Suppose the map document object is represented by variable mxDoc pointing to IMxDocument interface and the Transportation map is the second map, you can get the map from the document by the following line

```
Dim tranMap As IMap = mxDoc.Maps.Item(1)
```

and then activate the transportation map by

```
mxDoc.ActiveView = tranMap    ' QI is implicit, or
mxDoc.ActiveView = TryCast(tranMap, IActiveView) ' explicitly cast interface
```

2. Definition Query

The City GIS Department frequently receives requests from clients such as residents, realtors, and the assessor’s office to create maps for specific parcels. To create a map of a specific parcel based on the APN (Assessor’s Processing Number) provided by the client, a GIS specialist manually performs a definition query (Figure 3) and then zoom into the selected parcel.

Definition Query is to display a subset of features in a feature layer selected based on a query expression. An example of the result of a definition query is illustrated in Figure 4. This process can be automated using an add-in button which, when clicked, prompts for an APN number. Once an appropriate APN is received, the add-in automatically performs a definition query based on the APN and zooms into the selected parcel.

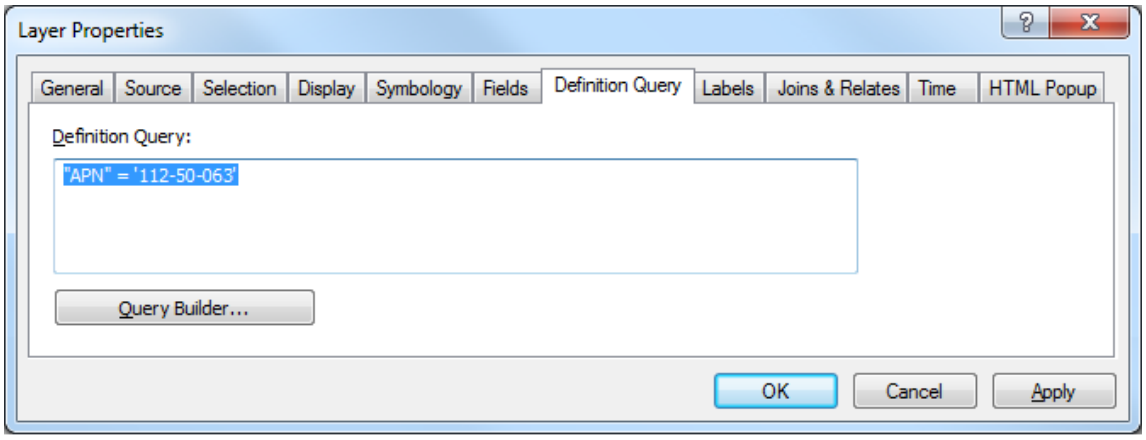


Figure 3 Manual definition query using ArcMap

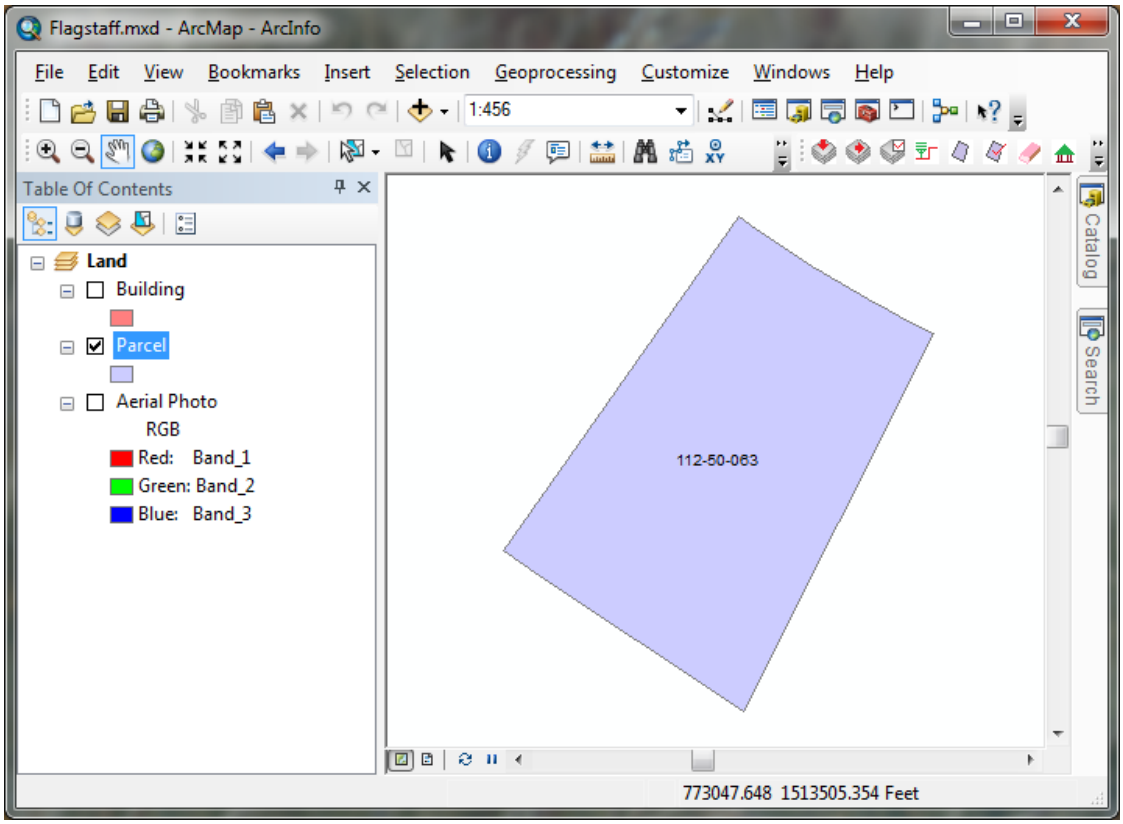


Figure 4 Result of a definition query

In ArcObjects programming, definition query can be accomplished by assigning a query expression to the DefinitionExpression property on the IFeatureLayerDefinition interface of the FeatureLayer class (Figure 5). Suppose you have a feature layer object with the IFeatureLayer interface, you can switch to the IFeatureLayerDefinition interface by QI.

A query expression is a text string such as “APN = ‘112-50-063’” where APN is a field name in the attribute table, and ‘112-50-063’ is a string value. This query expression is used to select a parcel whose APN code is ‘112-50-063’.

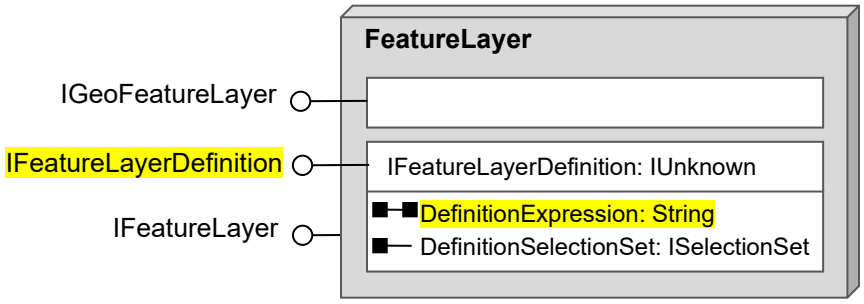


Figure 5 The DefinitionExpression property on the IFeatureLayerDefinition of the FeatureLayer class (included in Carto assembly)

To display parcel '112-50-063' only in ArcMap as shown in Figure 4 using programming, we can use the following code in the Click event procedure of an ArcMap add-in button:

```
Dim mxDoc As IMxDocument = My.ArcMap.Document  ' gets the map document
Dim map As IMap = mxDoc.Maps.Item(0)           ' gets the top map (index=0)
Dim lyr As ILayer = map.Layer(1)               ' gets the second layer (index=1)
Dim lyrDef As IFeatureLayerDefinition = lyr    ' QI to IFeatureLayerDefinition
lyrDef.DefinitionExpression = "APN = '112-50-063'"  ' sets a query expression

Dim av As IActiveView = map     ' QI, gets IActiveView of map
av.Refresh()                    ' refreshes map
```

A simple query expression is quite straightforward. It is a relational operation that compares an attribute with a value. The following are some rules about query expressions.

1) A query expression is a string or can be evaluated and results a string
2) If the entire expression is a simple string (which contains no variables), it must be quoted in a pair of double quotes
3) If the value in an expression is a string constant, such as `'112-50-063'`, the value must be quoted in a pair of single quotes. For example `"APN = '112-50-063'"`
4) If the value in an expression is a number or a Boolean value (True or False), the value must not be quoted. For example, `"Area > 10000"`
5) If a variable or text box is used to represent a **string value** in a query expression, the variable or text box must be concatenated to the expression string with single quotation marks placed around it in the expression. For example,
   - If a variable named *strAPN* holds an APN value (which is text), then a query expression must be written as `"APN = '" & strAPN & "'"`.
   - If a text box named *txtAPN* contains an APN value, then a query expression must be written as `"APN = '" & txtAPN.Text & "'"`.

   The single quotation marks placed on both sides of the string variable and the text box are included in the expression as characters.
6) If a variable or text box is used to represent a **numeric value** in a query expression, the variable or text box must be concatenated to the expression strings without single quote marks around it. For example,
   - If a variable named *pclArea* carries an area (numeric) value, then a query expression may be written as `"AREA >= " & pclArea`
   - If a text box named *txtArea* is used to receive a user-input parcel area, a query expression may be written as `"AREA >= " & txtArea.Text`

7) Relational operator **LIKE** can be used with wildcard symbols * (asterisk) to select records whose attribute values contain specified search keywords. For example,
   - To find street segments whose BASENAME attribute start with “BUT”, we can use query expression `"BASENAME LIKE 'BUT*'"` (note: one wildcard is placed after the search keyword).
   - To find street segments whose BASENAME attribute contains “FOR”, we can use query expression `"BASENAME LIKE '*FOR*'"` (note: a wildcard is placed before and after the search keyword).
8) Logical operators **And**, **Or**, and **Not** can be used to combine simple relational operations to make complex query expressions. For example,
   - To find street segments whose BASENAME attribute starts with “BUT” and SPEED attribute less than or equal to 25, a query expression can be written as "BASENAME LIKE 'BUT*' And SPEED <= 25".
   - To select parcels whose area is greater than 10000 and less than 100000, a query expression can be written as “Area > 10000 And Area < 100000”.

Example 2: To create an ArcMap add-in button for parcel mapping, please perform the following steps:

1) Create an add-in button in the CityGIS project. Name the button “MapParcelButton”, use “Map a parcel” for caption and tooltip text, and image  in folder CityGIS\Images on CD-ROM. Add the button into the CityGIS toolbar.
2) Bring up the code editor of the new add-in button.
3) Import the ArcMapUI and Carto namespaces into the class module.
4) Implement the OnClick() event procedure of the MapParcelButton as the following:

```
Protected Overrides Sub OnClick()
    ' use an input box to get an APN code from the user
    Dim strAPN As String = InputBox("Enter the APN of a parcel to map:" & Chr(10) & _
                                    "(enter nothing to display all features)", _
                                    "Parcel Mapping", "112-50-063")

    Dim strQuery As String = ""       ' declares a variable for query expression
    If strAPN <> "" Then              ' if the user input is not empty
        strQuery = "APN = '" & strAPN & "'"     ' creates a query expression
    End If

    Dim mxDoc As IMxDocument = My.ArcMap.Document
    Dim map As IMap = mxDoc.Maps.Item(0)          ' gets the top map (index=0)

    map.Layer(0).Visible = False                  ' turns off the building layer
    map.Layer(2).Visible = False                  ' turns off the image layer

    Dim lyr As ILayer = map.Layer(1)              ' gets the parcel layer (index=1)
    Dim lyrDef As IFeatureLayerDefinition = lyr   ' QI to IFeatureLayerDefinition
    lyrDef.DefinitionExpression = strQuery        ' sets query expression

    Dim av As IActiveView = map     ' QI to the IActiveView interface of map
    av.Refresh()                    ' refreshes map
    mxDoc.UpdateContents()
End Sub
```

VB built-in function InputBox() is used in the first line to prompt for an APN. Three string arguments are passed to the function. The first argument is a message that requests for a user

input. A long string is passed to this parameter. VB function Chr() is used to convert ASCII code 10 (line break) to a line break character (invisible) to break the long sentence into two lines. The second argument passed to the InputBox() function is a title for the box, and the third argument is a default value to appear in the input box. After prompting for a user input, the If block in the procedure checks if the user input is not empty. If not, it creates a query expression and preserves the expression in variable *strQuery*. If the input box is empty, the query expression variable stays blank. It is important to note that if the DefinitionExpression property is set with a blank string, the layer displays all features instead of nothing. After implementing the OnClick event of the button, definition query should work except that it does not zoom in to the selected parcel. You will gradually complete functions of this button.

To understand why a pair of single quote marks must be placed around a string variable in a query expression, you can set a break point at the "End If" line. When the program stops at the point, hover your mouse cursor over variable strQuery in the previous line to check its value. Then you may remove the quoted single quote marks and do the same to check the value of variable strQuery. You will see the difference.

3. Selecting Features

In ArcMap, when features are selected by whatever means, such as using the feature selection tool or querying attribute values, they are highlighted in the map. In ArcObjects programming, selecting features can be carried out by calling the SelectFeature() method on the IFeatureSelection interface of a feature layer object (Figure 6). Therefore, the process is similar to performing definition query, i.e. to call a method on another interface of a feature layer object.

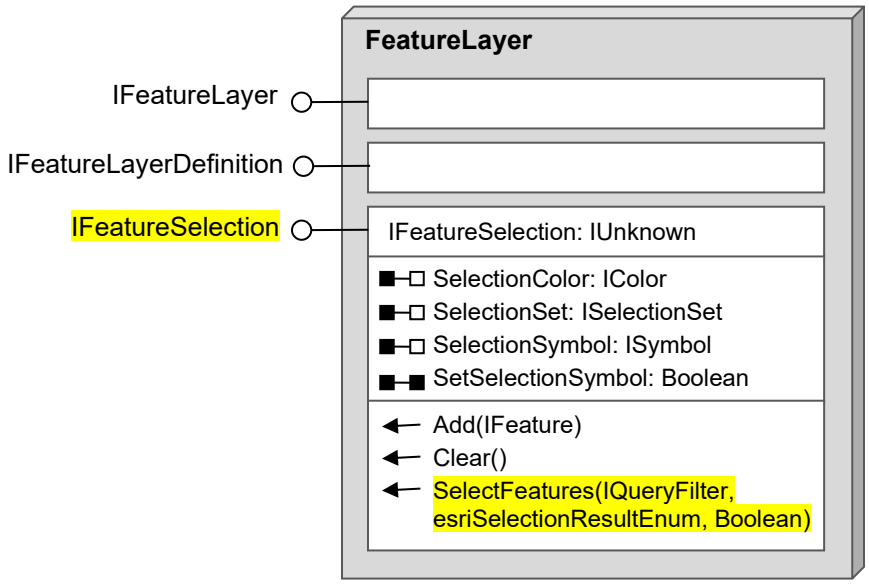


Figure 6 The IFeatureSelection interface of the FeatureLayer class
(FeatureLayer class is included in the Carto assembly)

The SelectFeatures() method requires three arguments: 1) a query filter object with IQueryFilter interface; 2) a constant selected from the ESRI selection result enumeration to indicate how to handle selected records; and 3) a Boolean value (True or False) to indicate whether to select only one feature if multiple features satisfy the selection criteria. While the last argument is

straightforward and you can specify “False” in most circumstances, the second argument is also easy to handle because the IntelliSense displays a list of options for you to select. Constant values included in this enumeration are shown in Figure 7.

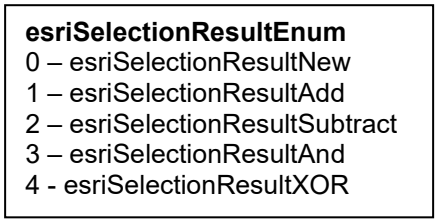


Figure 7 Constants of the esriSelectionResultEnum enumeration

In many circumstances, you may simply select the first option (0 – esriSelectionResultNew). Names of all options in the enumeration are self-explanatory.

The first parameter of the SelectFeatures() method, a query filter object, defines feature selection criteria. This object can be instantiated from the QueryFilter co-class (Figure 8).

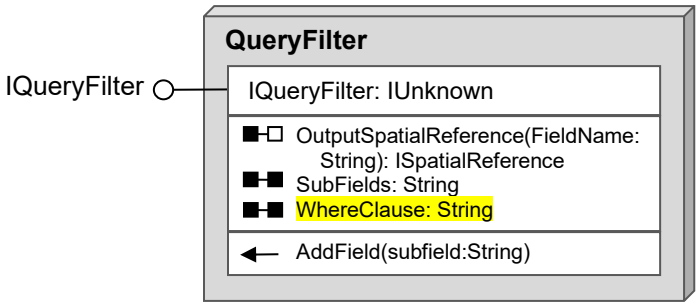


Figure 8 The QueryFilter co-class
(Class QueryFilter is included in the Geodatabase assembly)

The most important property to set to a new query filter object is the WhereClause property. This property accepts a query expression that defines feature selection criteria. A query expression to set to this property is exactly same as one you may use for a definition query. To select parcels that satisfy criteria “OWNERCLASS = ‘Private’ and SHAPE_AREA > 20000”, for example, you can perform the following steps:

1) Get the Land map and then the Parcel layer

```
Dim mxDoc As IMxDocument = My.ArcMap.Document    ' gets the document
Dim map As IMap = mxDoc.Maps.Item(0)             ' gets the Land map (top map)
Dim layer As IFeatureLayer = map.Layer(1)        ' gets the Parcel layer
```

2) Instantiate a query filter object and set its WhereClause property

```
Dim qFilter As IQueryFilter = New QueryFilter() ' creates a query filter object
' set the WhereClause property
qFilter.WhereClause = "OWNERCLASS = 'Private' And SHAPE_AREA > 20000"
```

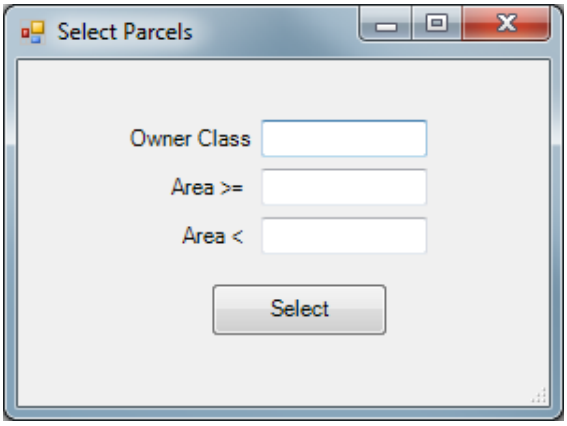
3) QI to the IFeatureSelection interface of the layer and call the SelectFeatures method

```
' QI to the IFeatureSelection interface
Dim fs As IFeatureSelection = layer
' call the SelectFeatures method
fs.SelectFeatures(qFilter, esriSelectionResultEnum.esriSelectionResultNew, False)
```

4) Refresh the map view

Example 3: To implement the parcel selection function, please perform the following steps:

1) Create an add-in button named "SelectParcelsButton". Use caption and tooltip text "Select Parcels" and image: in folder CityGIS\Images on CD-ROM. Add the button into the CityGIS toolbar.
2) Create a Windows Form (note: not ArcGIS dockable window) in your CityGIS project, and design the user interface with control names given in Figure 9.



Control	Name
Form	frmSelectParcels
TextBox	txtOwnerClass
TextBox	txtArea1
TextBox	txtArea2
Button	btnSelect

Figure 9 User interface of the parcel selection window

3) Add code to the OnClick event procedure of the add-in button (SelectParcelsButton) to display the windows form. You can run the program to test the button and make sure it opens the windows form. Stop running the program after the test.

```
Dim frmParcel As New frmSelectParcels     ' creates a form object from the class
frmParcel.ShowDialog()                    ' displays the form as a dialog
```

4) Open the code module of the Windows form (frmSelectParcels), and add the following imports statements into the header of the module.

```
Imports ESRI.ArcGIS.ArcMapUI          ' needed by Document
Imports ESRI.ArcGIS.Carto             ' needed by Map and Layer etc.
Imports ESRI.ArcGIS.Geodatabase       ' needed by QueryFilter
```

5) Add the following function into the frmSelectParcels class. This function creates a query expression from user input in the text boxes.

```
Private Function CreateQueryExpression() As String
    Dim str As String = ""

    If txtOwnerClass.Text <> "" Then
        str = "OWNERCLASS = '" & txtOwnerClass.Text & "'"  ' creates an expression
    End If

    If IsNumeric(txtArea1.Text) Then
        If str.Length > 0 Then str &= " And "      ' appends logical operator And
        str &= "SHAPE_AREA >= " & txtArea1.Text    ' adds a relational expression
    End If
```

```
        If IsNumeric(txtArea2.Text) Then
            If str.Length > 0 Then str &= " And "
            str &= "SHAPE_AREA < " & txtArea2.Text
        End If

        Return str
    End Function
```

6) Finally, implement the click event procedure of the selection button (btnSelect) as the following:

```
Private Sub btnSelect_Click(...) Handles btnSelect.Click
    Dim mxDoc As IMxDocument = My.ArcMap.Document   ' gets the document
    Dim map As IMap = mxDoc.Maps.Item(0)            ' gets the Land map
    Dim layer As IFeatureLayer = map.Layer(1)       ' gets the Parcel layer

    Dim qFilter As IQueryFilter = New QueryFilter() ' creates a query filter object
    qFilter.WhereClause = CreateQueryExpression()   ' sets the WhereClause property

    ' QI to the IFeatureSelection interface
    Dim fs As IFeatureSelection = layer
    ' call the SelectFeatures method
    fs.SelectFeatures(qFilter, esriSelectionResultEnum.esriSelectionResultNew, False)

    Dim av As IActiveView = map
    av.Refresh()
End Sub
```

Run the program to test the add-in button and the parcel selection functionality. For example, you may try to select parcels of owner class “Private” and area greater than 10000 and less than 30000. To understand how the CreateQueryExpression() function works, you can set a break point at the beginning of the function and step through the code line by line use the F8 key. Keep checking the value of variable *str* when you step over the function.

This program should work if the map and the parcel layer exist in the order shown in Figure 1. Otherwise, if the parcel layer does not exist or does not appear as the second layer (layer index 1) of the top map (map index 0), the program can crash. For code simplicity and demonstration clarity, the validity of map and layer are not checked in the above implementation. I would like to leave this tasks (to strengthen reliability of the program) for you to accomplish.

Finally, you can customize the color and symbol of selected features in map display. The IFeatureSelection interface has a SelectionColor property (see Figure 6) that allows you to set a custom color to render selected features. If you want to customize selected feature more, you can create a symbol based on the shape type (point, line, or polygon) of the layer and set it to the SelectionSymbol property, and then set the SetSelectionSymbol property to True.

4. SelectionSet and Cursor

Selection set and cursor are two container objects that hold selected features in ArcMap (Figure 10). A major difference between the two is that a selection set object does not support access to individual features but a cursor does. A cursor can be regarded as a table. Each row (or

record) is a feature that can be retrieved by the user. Therefore, a cursor is a more powerful container then a selection set.

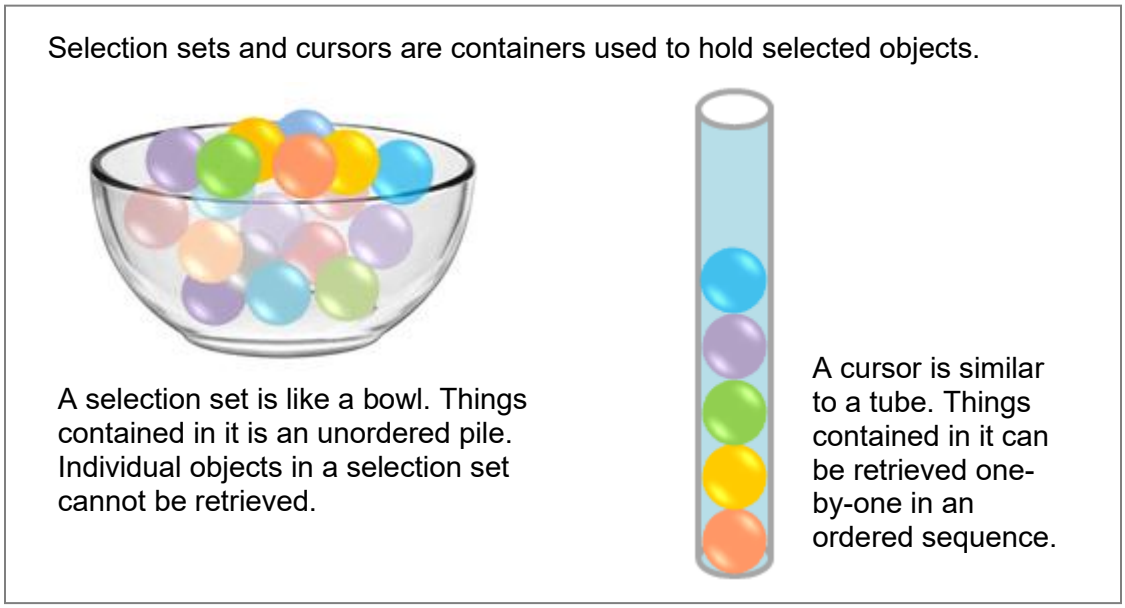


Figure 10 Selection set and cursor

4.1 SelectionSet

When features are selected in ArcMap by whatever means, they are stored in a selection set container object. The following are various methods that can select features in ArcMap.

- Use the select feature tool of ArcMap to click on features in the map
- Use the Select By Attributes tool of ArcMap to query features
- Use the SelectFeature() method on IFeatureSelection interface of a feature layer object to select features in a program (you just learned it in section 3)
- Use the Select() method on the IFeatureClass interface of a feature class object to select features in a program

In addition to the SelectFeature() method, the IFeatureSelection interface of the FeatureLayer class also provides properties to display selected features with user-specified colors or symbols (Figure 6). It also has a method to clear selected features in a selection set.

The selection set object can be obtained from ArcMap in two different approaches.

- To obtain it from the SelectionSet property on the IFeatureSelection interface of a feature layer object (see Figure 11). You can use this approach if you want to use features already selected in ArcMap.
- To obtain it by calling the Select() method on the IFeatureClass interface of a feature class object to generate selected features that satisfy certain criteria (also see Figure 11). You can use this approach if you want to query features from a feature class and get a selection set simultaneously.

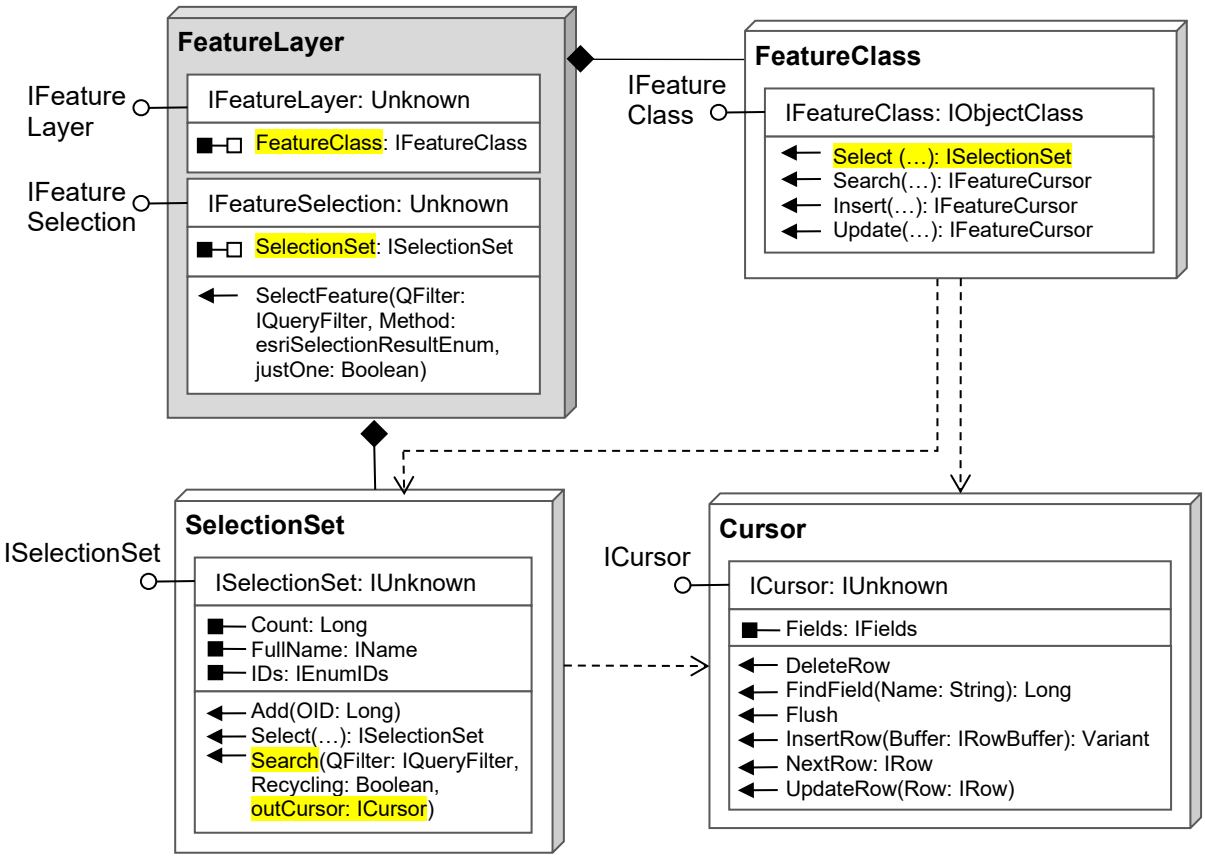


Figure 11 Model diagram for obtaining SelectionSet and Cursor objects
(Details on page 1 of GeoDatabaseObjectModel.pdf)

To use the second approach, the Select() method has four parameters (Figure 12):

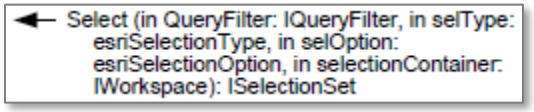


Figure 12 Parameters of the Select method

1) A query filter object introduced in Section 3.
2) A selection type that specifies where to store selected features' IDs. You can select a type from a built-in enumeration list.
   a. esriSelectionTypeIDset – store FIDs on the hard drive, good for large selection sets
   b. esriSelectionTypeSnapshot – store FIDs in memory, good for small selection sets
   c. esriSelectionTypeHybrid – let ArcGIS to decide. You can always choose it.
3) A selection option that specify how to handle multiple features satisfying query criteria. You can also select one from a built-in enumeration list.
   a. esriSelectionOptionNormal – select all features that meet query criteria
   b. esriSelectionOptionOnlyOne – select only one feature that meets query criteria
   c. esriSelectionOptionEmpty – do not select any

4) A workspace for storing the table (for selected FIDs) created by the second argument. You may pass value Nothing to it so that the selected IDs table is placed in the same workspace as the feature class (layer)

Suppose you have a feature layer object represented by variable *flayer*, you can create a selection set object by calling the Select() method using the following code:

```
' get the feature class from the feature layer
Dim fclass As IFeatureClass = flayer.FeatureClass

' create a new query filter object
Dim qfilter As IQueryFilter = New QueryFilter
qfilter.WhereClause = (a query expression ...)

' call the Select() method to create a selection set object
Dim selset As ISelectionSet = _
    fclass.Select(qfilter,
                  esriSelectionType.esriSelectionTypeHybrid,
                  esriSelectionOption.esriSelectionOptionOnlyOne,
                  Nothing)
```

4.2 Cursor

A cursor is a more powerful container object for holding selected features than the SelectionSet because it provides access to individual features. Once you obtain an individual feature from a cursor, you can access its spatial and attribute data. Being able to access the spatial and attribute data of individual features allows you to develop your own spatial analysis and spatial modeling tools.

Figure 11 shows one property and a number of useful methods of the Cursor class. The Fields property provides information, including field name, data type, length, precision, etc., of all fields in the attribute table. The Cursor class provides methods to retrieve individual features (records / rows), as well as to insert and delete features. A Cursor object can also be obtained by several different approaches.

First, if ArcMap contains some selected features in a selection set, we can obtain a cursor object by calling the Search() method on the ISelectionSet interface of the selection set object (Figure 13). The Search() method returns a cursor object with ICursor interface via a ByRef parameter (the third argument, see Figure 11).

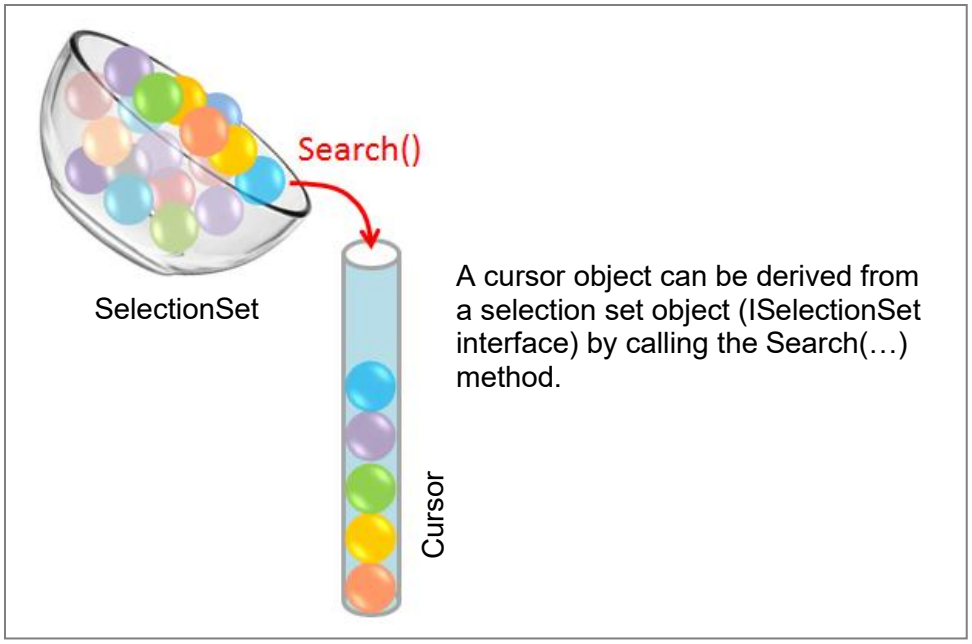


Figure 13 Obtaining a cursor form a selection set

Second, the IFeatureClass interface of the FeatureClass class has three methods, Search(), Insert() and Update(), each returning a cursor object with the IFeatureCursor interface (Figures 11 and 14).

- Search(queryFilter: IQueryFilter, recycling: Boolean): Good for obtaining shapes or attribute values from a feature class for display or information query purpose. It may also be used for editing small number of selected features.
- Update(queryFilter: IQueryFilter, recycling: Boolean): Good for editing spatial and attribute data in a feature class, especially for editing large number of records or the entire feature class.
- Insert(useBuffering: Boolean): Use it when you want to insert new features (records) into a feature class.

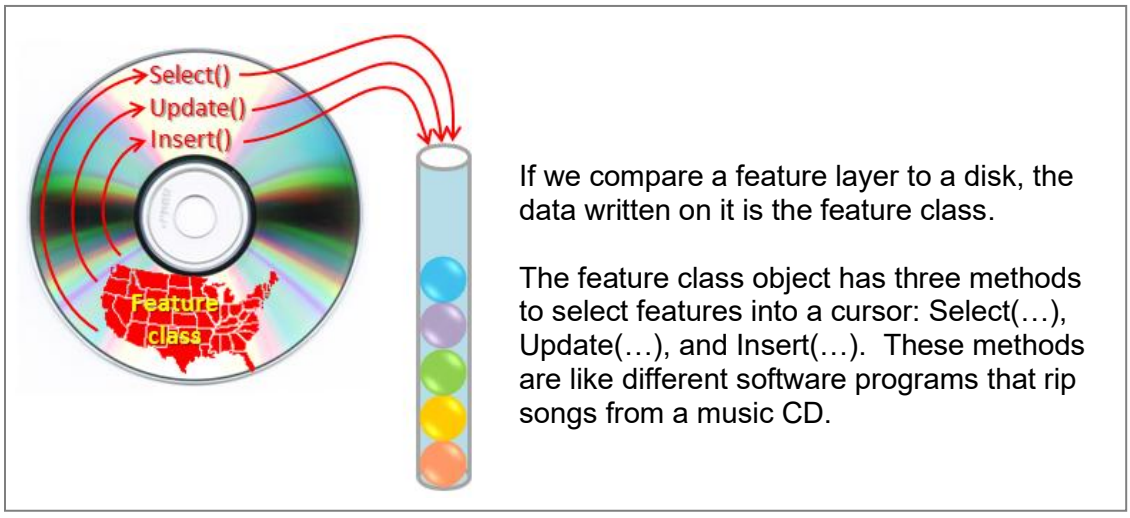


Figure 14 Obtaining cursors from a feature class

The Search() and Update() methods have two parameters: a query filter object, and a Boolean value. If you pass value Nothing to the query filter parameter, all features in the feature class (layer) will be selected into the cursor. The second parameter controls whether to rehydrate a single feature object on each fetch when using the NextFeature() method. If parameter recycling is set to True, one address is repeatedly used to store features retrieved from the cursor in each fetch. Therefore, value True can be used to optimize read-only access such as to draw features. If parameter recycling is set to False, each feature extracted from the cursor will be stored in a unique address. Therefore, value False should be used when you edit features. The Insert() method has one parameter that indicates whether to use buffering or not. You can normally use True for this argument.

Given a feature layer object represented by variable *flayer*, you can use the Search() method to obtain a cursor containing selected features from its feature class using the following code:

```
' get the feature class from the feature layer
Dim fclass As IFeatureClass = flayer.FeatureClass

' create a new query filter object
Dim qfilter As IQueryFilter = New QueryFilter
qfilter.WhereClause = (query expression ...)

' call the search() method to create a cursor
Dim fcursor As IFeatureCursor = fclass.Search(qfilter, True)
```

5. FeatureCursor and Feature

The FeatureCursor (regular) class inherits the Cursor class, which means a feature cursor is a kind of cursor (table). But a feature cursor also contains feature geometry data in a table. When you get a cursor from a non-spatial table, you get a collection of selected records (or rows); when you get a cursor from a feature class, you get a collection of features in table format which is a feature cursor.

A feature cursor has a very similar set of methods and properties as a general cursor (Figure 15) except that word “Row” in methods of a cursor are replaced by “Feature”. The NextFeature() method returns a feature object with IFeature interface. By repeatedly using this method in a loop, we can obtain every individual feature in a feature class.

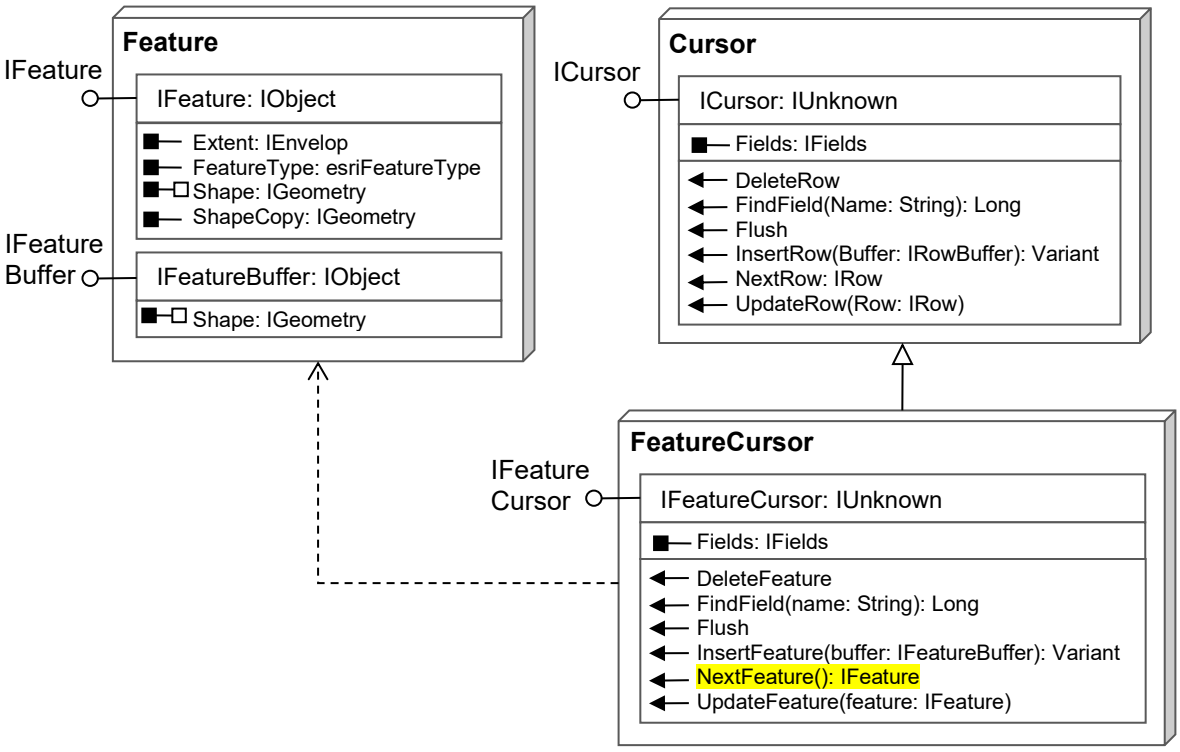


Figure 15 Cursor, FeatureCursor and Feature regular classes
(Details on page 1 of GeoDatabaseObjectModel.pdf)

A feature is an object composed of a geometric shape and associated attribute data. The IFeature interface of the Feature (regular) class provides several read-only properties for the user to access geometric information: the ShapeType property provides information about the geometric type (such as point, polyline, or polygon) of the feature; the Extent property returns an envelope object (with IEnvelope interface) that defines the extent of the feature; and the read-only ShapeCopy property returns a non-editable geometric shape of the feature. The IFeature interface also has a two-way property Shape which allows for reading and setting geometry.

In addition to the IFeature interface, the Feature class has many other interfaces and it inherits more interfaces so that it provide a large number of properties and methods for accessing and manipulating spatial and attribute data. In the rest of this chapter, you will learn how to use one of the properties on the IFeature interface – the Extent property. More properties and methods will be introduced in Chapter 13.

Example 4: In Section 2 of this chapter, you created definition query on the parcel feature layer so that one parcel of a specified APN is displayed for parcel mapping. But you did not complete the zoom functionality that automatically zooms the map into the selected parcel. To implement this functionality, you can create a sub procedure, name it ZoomToFeature(), to carry out the task. The sub procedure must declare two parameters, a feature layer object and a query expression. The following is a general procedure for implementing the ZoomToFeature() procedure.

1) Get the feature class object from the feature layer

2) Use the client provided query criteria to set up a query filter, and then call the Search() method to create a feature cursor object from the feature class.
3) Call the NextFeature() method to retrieve the selected parcel feature from the feature cursor.
4) Get an envelope object from the Extent property of the selected feature
5) Set the envelope object to the Extent property on the IActiveView interface of the map to zoom to the parcel feature

Before implementing the ZoomToFeature()procedure in the MapParcelButton class, please add the following two lines into the header of the MapParcelButton class module to import necessary namespaces:

```
Imports ESRI.ArcGIS.Geodatabase
Imports ESRI.ArcGIS.Geometry
```

The Geodatabase namespace is needed by the FeatureClass and QueryFilter classes and the Geometry namespace is required by the Envelope class. Once these two namespaces are imported, you can implement the ZoomToFunction() sub procedure as the following.

```
    Private Sub ZoomToFeature(flayer As IFeatureLayer, query As String)
        If query = "" Then Return

        ' 1) get the feature class from the feature layer
        Dim fclass As IFeatureClass = flayer.FeatureClass

        ' 2) create a feature cursor from the feature class
        ' a. create a new query filter object
        Dim qfilter As IQueryFilter = New QueryFilter
        qfilter.WhereClause = query

        ' b. call the search() method to create a cursor
        Dim fcursor As IFeatureCursor = fclass.Search(qfilter, True)

        ' 3) get a feature from the feature cursor by calling the NextFeature method
        ' note that there is only one feature in the cursor
        Dim feature As IFeature = fcursor.NextFeature

        ' 4) get the envelope object from the Extent property of the feature object
        Dim ext As IEnvelope = feature.Extent

        ' expand the envelope by 10% so that the feature
        ' won't touch map borders after zooming in
        ext.Expand(1.10, 1.10, True)

        ' 5) set the envelop to the Extent property of the map to zoom in.
        Dim av As IActiveView = My.ArcMap.Document.ActiveView
        av.Extent = ext
    End Sub
```

In step 2) a query filter object was created first because the Search() method needs it. In step 4) the envelope object is purposely expanded by factor 1.1 to create a 10% white space around the selected parcel and prevent the parcel boundaries from touching the map borders when the map is zoomed in step 5).

Finally, you can call the sub procedure at the end of the OnClick() event procedure of add-in button MapParcelButton before refreshing the map (above Dim av As …):

```
ZoomToFeature(lyr, strQuery)    ' call the ZoomToFeature() sub procedure
```

Run the program to test the map parcel add-in button in ArcMap. You may test with these two APNs: 112-50-063, 112-51-135.

Summary and Highlights

A map document can maintain multiple maps (data frames). The IMxDocument interface of a document object has a Maps property that returns a maps object. A maps object is a container that holds map objects. You can retrieve a map object from the Item property of the maps object by passing an index to the property. The Map class is a co-class that can instantiate new map objects. Once a map object is instantiated, you can assign a name to it, add layers into it, and add the map into the maps container.

Definition query is to use a query expression to select a subset of features from a feature layer to display and filter out the rest features. To perform a definition query in program is to set a query expression to the DefinitionExpression property on the IFeatureLayerDefinition interface of a feature layer object.

A query expression is a string of logical / relational operations in which certain fields of a feature class are compared with specific values. For a numeric or Boolean field, the value must not be quoted by single quote marks in the expression string. For a string field, the value must be quoted by single quote marks. If the value of string field is hold by a variable, the variable must be concatenated in the expression string, and single quote marks must be used around the variable in the expression string.

To select features from a feature layer by a query expression, you must create a query filter object and pass it, along with two other arguments, to the SelectFeatures() method on the IFeatureSelection interface of a feature layer object. A query filter object can be instantiated from co-class QueryFilter. Once a query filter object is created, you can set the query expression to its WhereClause property.

In ArcMap, when a subset of features are selected from a feature layer, they are hold by the selection set object. A selection set is a container in which selected objects are unordered and cannot be retrieved. But it provides properties to change the display of the features such as color and symbol for display.

A cursor is a more powerful container that holds features selected by programs. Objects selected into a cursor are ordered and can be retrieved one-by-one. A cursor can be obtained from a feature class object by calling the Search(), Insert(), or Update() method. Cursors derived from different methods have different intended purposes. A feature cursor is a kind of cursor that holds selected features and it supports retrieval of individual features by the NextFeature() method.

A feature object provides properties for developers to access its spatial and attribute data. The ShapeCopy property returns a read-only copy of the geometry of the feature and the Shape property allow two-way access to the feature’s geometry.

Exercises

1. Answer the following questions
   1) How do you manually switch between data frames in ArcMap?
   2) How do you get a specific map object from a document by programming?
   3) How do you create a new map and add it into the Document by programming?
   4) How do you activate a specific map in the Document by programming?
   5) What is definition query? How do you manually perform definition query in ArcMap?
   6) Given a feature layer object, what interface is the DefinitionExpression property on?
   7) What is a query expression?
   8) What values in a query expression must be quoted by single quote marks?
   9) Do we need to quote a numeric value in a query expression by single quotes?
   10) How do you place a variable carrying a string value in a query expression?
   11) How do you place a variable carrying a numeric value in a query expression?
   12) How do you use wildcards in a query expression?
   13) How do you use the InputBox to get a user input?
   14) What interface of a feature layer object does the SelectFeature() method reside on?
   15) What is a query filter? How do you create a query filter?
   16) What is the WhereClause property of a query filter used for?
   17) In ArcMap, what object preserves selected features?
   18) What is the SelectionSet? What is a Cursor? What is the main difference between them?
   19) How can features be selected into the SelectionSet?
   20) How do you obtain the SelectionSet object from ArcMap by programming?
   21) What method do you call to get a cursor from a selection set object?
   22) What are the three methods on the IFeatureClass interface that produce feature cursors?
   23) What method do you call to retrieve a feature object from a feature cursor?
   24) How do you find the geometry type of a feature?
   25) How do you obtain a geometric shape of a feature?
   26) How do you zoom a map to the extent of a given envelope object?

2. In ArcObjects programming, selecting features from a feature layer requires you to create a query expression and set it to the WhereClause property of a query filter object. Given STATE_NAME (for state name) and AVG_SALE87 (for average home sale price in 1987) are two fields in the attribute table of the US Counties feature layer. To select counties that satisfy certain criteria, please write a query expression for each question below in Windows **Notepad**.

   ```
   Dim strQuery As String
   ```

   1) Select all counties in Wisconsin:
      ```
      strQuery =
      ```

   2) Select all counties in the US whose average home sale prices in 1987 were greater than or equal to 75000:
      ```
      strQuery =
      ```

   3) Select all counties in Wisconsin and Illinois:
      ```
      strQuery =
      ```

4) Select counties in Wisconsin whose average home sale prices in 1987 were greater than or equal to 75000:
```
strQuery =
```

5) In a Windows form, there is a combo box named cboState which contains all state names of the US. Select all counties in a state shown in the combo box:
```
strQuery =
```

6) In a Windows form, there is a text box named txtPrice which allows the user to enter a long integer number. Select all counties in the US whose average home sale prices in 1987 were greater than or equal to the value shown in the textbox:
```
strQuery =
```

7) In a Windows form, there is a combo box named cboState which contains all state names of the US and a text box named txtPrice which allows the user to enter a long integer number. Select all counties in the state indicated in the combobox **and** whose average home sale prices in 1987 were greater than the value in the textbox:
```
strQuery =
```

8) In a Windows form, there is a combo box named cboState which contains all state names of the US and a text box named txtPrice which allows the user to enter a long integer number. Select all counties in the state indicated in the combo box **or** whose average home sale prices in 1987 were greater than the value in the text box:
```
strQuery =
```

3. In Example 3 of this chapter, a functions was created in the CityGIS project to select parcels from the parcel feature layer. The program should work if maps and layers in ArcMap are organized as required by the program, i.e. the parcel layer must appear as the second layer of the top map in the document. However, the program is not robust because if layer are not organized this way, the program will crash.

   To enhance the program, please write a function in the frmSelectParcels class module to find any specific layer with a given layer name in the document. The function should return a feature layer object with IFeatureLayer interface. The following are a few requirements and tips:

   - Name the function GetLayer() and declare a string parameter to receive a layer name as input. The function must return a feature layer object (with IFeatureLayer interface). Therefore, the declaration of the function can be as the following.

     ```
     Private Function GetLayer(LayerName As String) As IFeatureLayer

     End Function
     ```

   - To find the layer of the given name, you need to use two nested repetitions to examine all layers in all maps in the document. Use the object model diagram of Figure 2 to help figure out how to iterate through all maps as well as layers in each maps. Once the name of a layer matches the specified layer name, the function returns that layer. If no layer matches the criterion after executing all iterations, use code “Return Nothing” in the end of the function.

- Call the function to get the parcel layer in the click event procedure of the select button (btnSelect) to replace the two lines of code that get the parcel layer with hard-coded map and layer indexes.

```
Dim layer As IFeatureLayer = GetLayer("Parcel")
```